TDD (테스트 주도 개발) 학습 노트

날짜: 2026-05-22
복습 예정: D+1, D+3, D+7

한 줄 요약

코드 짜기 전에 테스트 먼저 쓰는 개발 방식 — "이렇게 동작해야 한다"를 먼저 정의하고 코드를 맞춰간다.

---

1. 비유

일반 개발:

```
코드 작성 -> 실행해서 확인 -> 버그 발견 -> 수정 -> 반복
```

TDD:

```
테스트 작성 (실패) -> 코드 작성 (테스트 통과) -> 리팩토링 -> 반복
```

자동차 설계에 비유:

- 일반: 차 먼저 만들고 -> 충돌 테스트
- TDD: 충돌 기준 먼저 정의 -> 그 기준 통과하는 차 설계

---

2. 개념

```
TDD 사이클 (Red -> Green -> Refactor):
```

>

```
1. Red: 실패하는 테스트 작성
2. Green: 테스트를 통과하는 최소한의 코드 작성
3. Refactor: 코드 정리 (테스트는 계속 통과 상태 유지)
```

---

3. 왜 쓰나

- 없으면: 구현 후 테스트 -> 테스트가 구현에 맞춰짐 -> 잘못된 동작도 테스트가 통과
- 있으면: 테스트가 명세서 역할 -> 구현이 테스트에 맞춰짐 -> 올바른 동작 보장

Dev 2에서: 각 노드(run_tools, llm_judge 등)를 TDD로 구현 -> 노드 교체 시 회귀 버그 방지

---

4. 코드 (Dev 2)

예시: llm_judge_node Minor 스킵 TDD

```
[python]
# Step 1: RED — 실패하는 테스트 먼저 작성
# dev2/tests/test_nodes.py

import pytest
from unittest.mock import AsyncMock, patch

@pytest.mark.asyncio
async def test_llm_judge_skips_minor():
    from dev2.nodes.llm_judge import llm_judge_node  # 아직 없음 -> ImportError
    minor_issue = {
        "severity": "Minor", "line": 1, "file": "a.py",
        "message": "unused var", "rule_id": "SA-001",
        "suggestion": "fix", "similar_issues": []
    }
    state = {"payload": PAYLOAD, "enriched": [minor_issue], ...}

    with patch("dev2.nodes.llm_judge._get_cached_llm") as mock_llm:
        result = await llm_judge_node(state)
        mock_llm.assert_not_called()  # Minor는 LLM 호출 안 해야 함
        assert result["enriched"][0]["severity"] == "Minor"
```

```
[bash]
# 실행 -> FAIL (ImportError: No module named 'dev2.nodes.llm_judge')
pytest dev2/tests/test_nodes.py::test_llm_judge_skips_minor -v
```

```
[python]
# Step 2: GREEN — 테스트 통과하는 최소 코드 작성
# dev2/nodes/llm_judge.py

async def llm_judge_node(state):
    result = []
    for issue in state["enriched"]:
        if issue["severity"] == "Minor":  # Minor 스킵
            result.append(issue)
            continue
        # ... LLM 호출
    return {"enriched": result}
```

```
[bash]
# 실행 -> PASS
pytest dev2/tests/test_nodes.py::test_llm_judge_skips_minor -v
```

```
[python]
# Step 3: REFACTOR — 코드 정리 (테스트는 계속 통과)
# MODEL_ROUTING dict로 정리, lru_cache 추가 등
```

---

5. pytest 핵심 문법

```
[python]
# 기본 테스트
def test_something():
```

```
    assert result == expected

# 비동기 테스트
@pytest.mark.asyncio
async def test_async():
    result = await some_async_func()
    assert result == expected

# Mock: 외부 의존성 대체
from unittest.mock import patch, AsyncMock

with patch("module.function", return_value="fake"):
    result = function_that_calls_it()

# AsyncMock: async 함수를 대체
with patch("module.async_func", new_callable=AsyncMock, return_value="fake"):
    result = await function_that_calls_it()
```

복습 퀴즈

1. Q: TDD Red-Green-Refactor 순서의 의미는?
   A: (Red=실패 테스트 작성, Green=최소 코드로 통과, Refactor=코드 정리)
2. Q: TDD에서 Mock을 쓰는 이유는?
   A: (LLM API 호출, DB 같은 외부 의존성 없이 단위 테스트 가능하게 하기 위해)
3. Q: Dev 2에서 TDD로 얻는 이점은?
   A: (각 노드의 동작 보장 + LangGraph 순서 변경 시 회귀 버그 즉시 감지)